

Протоколы взаимодействия: HTTP, REST, gRPC, GraphQL, WebSocket

Table of contents

Протоколы взаимодействия: HTTP, REST, gRPC, GraphQL, WebSocket	1
Прикладные протоколы взаимодействия	1
REST	1
gRPC	2
GraphQL	2
WebSocket	2

Протоколы взаимодействия: HTTP, REST, gRPC, GraphQL, WebSocket

В предыдущей лекции мы разобрали модель OSI, понятие протокола как договора и форматы спецификаций API (OpenAPI, Proto, GraphQL SDL). Теперь рассмотрим конкретные прикладные протоколы и их практическое применение.

Прикладные протоколы взаимодействия

Лекция рассматривает четыре прикладных протокола: REST, gRPC, GraphQL и WebSocket. Они решают разные классы задач: классический обмен «запрос-ответ», высокопроизводительные межсервисные удалённые вызовы процедур (RPC), гибкие выборки данных и двустороннюю коммуникацию в реальном времени. Выбор протокола определяется требованиями к производительности, удобству интеграции, гибкости модели данных и характеру пользовательского сценария.

REST

- **Модель взаимодействия:** запрос-ответ на основе HTTP-методов (GET, POST, PUT, DELETE) и кодов статуса.
- **Формат и транспорт:** обычно HTTP + JSON; ресурсы адресуются URI (например, `/api/users/123`).
- **Преимущества:** простота, широкая совместимость, удобная диагностика и развитая экосистема (Swagger/OpenAPI).
- **Ограничения:** избыточный объём данных и необходимость множества конечных точек API для разных сценариев представления.
- **Нюансы внедрения:** важно строго соблюдать семантику HTTP-методов, корректно использовать коды ответов, а также проектировать версионирование API (например, `/v1`) и политику обратной совместимости.

gRPC

- **Модель взаимодействия:** удалённые вызовы процедур (RPC), включая одиночные и потоковые вызовы.
- **Формат и транспорт:** Protocol Buffers как схема данных, HTTP/2 как транспорт.
- **Преимущества:** высокая производительность, компактная бинарная сериализация, строгая типизация контракта.
- **Ограничения:** более сложная отладка, обязательная кодогенерация и ограничения прямого использования в браузере (обычно через gRPC-Web).
- **Нюансы внедрения:** необходима дисциплина эволюции `.proto`-контрактов (зарезервированные поля и совместимость схем), настройка дедлайнов и метаданных, а также понимание инфраструктурных ограничений (балансировщики, прокси, наблюдаемость HTTP/2-трафика).

GraphQL

- **Модель взаимодействия:** клиент формулирует запрос как дерево полей и получает только требуемые данные.
- **Формат и транспорт:** чаще всего HTTP с единой точкой входа (`/graphql`); ответы обычно в JSON.
- **Преимущества:** уменьшение избыточной и недостаточной выборки данных (over-fetching/under-fetching) и удобная поддержка разных экранов и клиентских сценариев.
- **Ограничения:** более сложная эксплуатация (контроль сложности запросов, кэширование, авторизация на уровне полей).
- **Нюансы внедрения:** критично ограничивать глубину и вычислительную сложность запросов, предотвращать проблему N+1 через пакетирование запросов (batching/DataLoader) и явно проектировать границы ответственности слоя резолверов.

WebSocket

- **Модель взаимодействия:** постоянное двустороннее соединение между клиентом и сервером.
- **Формат и транспорт:** устанавливается через HTTP Upgrade, далее обмен сообщениями идёт по WebSocket-каналу.
- **Преимущества:** низкая задержка и серверные push-обновления без постоянного опроса.
- **Ограничения:** более сложное управление соединениями (повторное подключение, контроль активности соединения, регулирование входящего потока) и хранение состояния соединений на сервере.
- **Нюансы внедрения:** требуются протокол прикладных сообщений (типы событий, версии), механизмы восстановления соединения, защита от «медленных» клиентов и план масштабирования соединений с состоянием (маршрутизация с привязкой к узлу или внешний канал публикации/подписки).